



GIT & GITHUB

Controle de versão e colaboração na prática

Prática de Desenvolvimento de Sistemas

Fatec

Barueri

Padre Danilo

José de Oliveira Ohl

Objetivos da aula

- Entender por que usamos controle de versão.
- Diferenciar Git de GitHub e repositório local de remoto.
- Executar o ciclo add → commit → push.
- Trabalhar com branches, merge e Pull Requests.
- Reconhecer e resolver um conflito simples.

Antes do Git...

O problema que queremos resolver

- projeto.java
- projeto_final.java
- projeto_final_2.java
- projeto_final_agora_vai.java
- Qual é a versão certa? Quem alterou? Como voltar?

O que é Git?

- Sistema de controle de versão distribuído.
- Registra mudanças no projeto ao longo do tempo.
- Permite comparar, recuperar e combinar versões.
- Funciona localmente: Git não é sinônimo de GitHub.

Git × GitHub

- Git: ferramenta de versionamento instalada no computador.
- GitHub: serviço que hospeda repositórios Git na internet.
- Git controla o histórico; GitHub facilita compartilhamento e colaboração.
- Também existem outras plataformas para hospedar Git.

Salvar × Commit × Push

Três ações diferentes

SALVAR

Arquivo no computador

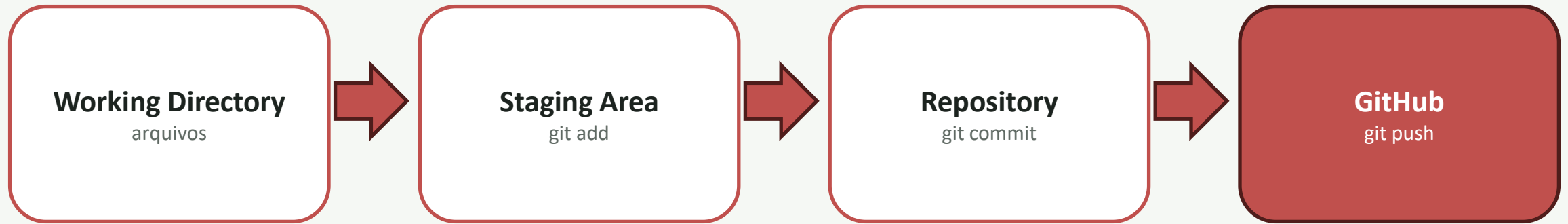
COMMIT

Versão no Git local

PUSH

Commits no GitHub

O ciclo básico do Git



Configuração inicial

Feita uma vez por computador

```
git --version
```

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "voce@email.com"
```

```
git config --list
```

Criando o primeiro repositório

```
cd aula-git
```

```
git init
```

```
git status
```

```
mkdir aula-git
```

Primeiro commit

crie ou altere um arquivo

```
git status
```

```
git add .
```

```
git status
```

```
git commit -m "Cria projeto inicial"
```

```
git log --oneline
```

O que é um bom commit?

- Pequeno e focado em uma mudança coerente.
- Mensagem que explica o que foi feito.
- Evite commits gigantes como “alterações” ou “coisas”.
- Ex.: “Adiciona validação de idade do aluno”.

.gitignore

- Define arquivos e pastas que não devem ser versionados.
- Em Java/Maven: target/, arquivos de IDE e segredos.
- Nunca versionar senhas, tokens ou chaves.
- O código-fonte deve ser reproduzível sem artefatos gerados.

Conectando ao GitHub

O repositório remoto passa a ser chamado de origin

```
git remote -v  
git branch -M main  
git push -u origin main
```

```
git remote add origin <URL-DO-REPOSITORIO>
```

Clone, Pull e Push

```
git clone <URL>
```

```
# receber alterações
```

```
git pull
```

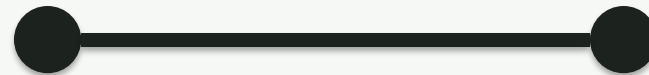
```
# enviar seus commits
```

```
git push
```

```
# copiar um repositório
```

Branches: linhas paralelas de trabalho

main



feature/cadastro-aluno

Criando uma branch

```
git switch -c feature/cadastro-aluno
```

```
# altere o código
```

```
git add .
```

```
git commit -m "Adiciona cadastro de aluno"
```

```
git push -u origin feature/cadastro-aluno
```

Merge local

`git pull`

`git merge feature/cadastro-aluno`

`git push`

`git switch main`

Pull Request (PR)

- Proposta para integrar alterações de uma branch em outra.
- Permite revisar código, conversar e executar verificações.
- O PR não substitui Git: ele é um fluxo de colaboração da plataforma.
- Fluxo comum: branch → commits → push → PR → review → merge.

Conflitos de merge

- Acontecem quando o Git não consegue decidir sozinho como combinar mudanças.
- São comuns quando duas branches alteram a mesma região de um arquivo.
- O desenvolvedor escolhe o conteúdo correto e conclui o merge.
- Conflito não significa que o repositório está quebrado.

Como o conflito aparece

```
System.out.println("Olá, turma!");
```

```
=====
```

```
System.out.println("Bem-vindos à aula!");
```

```
>>>>>> feature/mensagem
```

```
<<<<<<< HEAD
```

Resolvendo um conflito

1. edite o arquivo e escolha o conteúdo correto

```
git add src/main/java/...  
git commit
```

confira o resultado

```
git status  
git log --oneline --graph --all
```

Fluxo profissional simplificado

- 1. Atualize a main.
- 2. Crie uma branch para a tarefa.
- 3. Implemente e faça commits pequenos.
- 4. Faça push da branch.
- 5. Abra um Pull Request.
- 6. Revise, ajuste e faça merge.

Prática guiada — parte 1

- Clone o projeto Java disponibilizado.
- Crie uma branch feature/seu-nome.
- Implemente uma nova opção no menu.
- Faça pelo menos um commit com mensagem clara.
- Envie a branch para o GitHub.

Prática guiada — parte 2

- Abra um Pull Request para main.
- Leia as alterações na aba de diff.
- Peça revisão a um colega.
- Faça uma pequena correção e envie novo commit.
- Observe o PR sendo atualizado automaticamente.

Desafio em dupla

- Aluno A: adiciona a opção “Listar alunos”.
- Aluno B: adiciona a opção “Buscar aluno por nome”.
- Cada um trabalha em sua própria branch.
- Integre as duas mudanças via Pull Request.
- Bônus: provoque e resolva um conflito de propósito.

Comandos essenciais

- status • add • commit • log
- switch • branch • merge
- clone • pull • push
- remote
- O objetivo é entender o fluxo — não decorar comandos.

Erros comuns

- Trabalhar sempre diretamente na main.
- Fazer git add . sem conferir o git status.
- Commits enormes ou mensagens vagas.
- Dar pull só depois de terminar todo o trabalho.
- Versionar senhas, tokens, target/ ou arquivos desnecessários.

Mapa mental final

- Git registra versões localmente.
- GitHub hospeda e organiza a colaboração.
- add prepara; commit registra; push publica.
- branch isola trabalho; merge combina.
- Pull Request organiza revisão e integração.

Checklist: você consegue...?

- Criar ou clonar um repositório?
- Fazer add, commit, pull e push?
- Criar e trocar de branch?
- Explicar o que é um Pull Request?
- Resolver um conflito simples?



Git é histórico.

GitHub é colaboração.

Pratique o fluxo até ele
ficar natural.

Fatec

Barueri

Padre Danilo

José de Oliveira Ohl